

A REPORT

on

Industrial Training - I

Submitted in Partial Fulfillment of the requirements For the Degree of
Bachelor of Technology in CSE

By

Suraj Arya

CSE II Year, III Semester

Registration No.: 25105103904

Netaji Subhas Institute of Technology (Patna)



Under the Supervision of

Dean

Ms. J. Dalei
(Academic Dean)
NSIT, Patna

Coordinator

Mr. Ramakant Singh
(Head of Department)
NSIT, Patna



to the

Faculty of B.Tech

Bihar Engineering University (BEU), 2024-28



CERTIFICATE OF COMPLETION

This is to certify that the report entitled “**Industrial Training – I**” is a bonafide record of the work carried out by **Suraj Arya (Registration No.: 25105103904)**, a student of **Bachelor of Technology in Computer Science and Engineering (CSE)** during the academic session **2024–2028** in the **Second Year, Third Semester** at **Netaji Subhas Institute of Technology (NSIT), Patna**.

This report has been submitted in partial fulfillment of the requirements for the award of the **Bachelor of Technology (B.Tech)** degree under **Bihar Engineering University (BEU), Patna**.

The training was successfully completed at **C-DAC, Patna** on the topic “**OOP’s Concepts using Core Java**”.

Ms. J. Dalei
(Academic Dean)
NSIT, Patna

Mr. Ramakant Singh
(Head of Department)
NSIT, Patna

EXTERNAL EXAMINER

Date:

Acknowledgement

I take this opportunity to express my heartfelt gratitude to all those who have supported and guided me throughout my **Industrial Training – I**, enabling me to successfully complete this report entitled “**Industrial Training – I**”.

Firstly, I would like to extend my sincere thanks to **Netaji Subhas Institute of Technology (NSIT), Patna**, for providing me with the opportunity to undertake this training, which has been a valuable learning experience in my academic journey.

I would also like to express my sincere gratitude to **C-DAC, Patna**, for organizing this training program on “**OOP’s Concepts using Core Java**” and for providing excellent learning resources and practical exposure.

I am highly thankful to the respected faculty members and trainers, especially **Mr. Vaibhav Patle** and **Ms. Nishu**, for their continuous guidance, support, and valuable insights throughout the training period.

I would also like to thank **Ms. J. Dalei (Academic Dean)** and **Mr. Ramakant Singh (Head of Department)**, NSIT Patna, for their encouragement and support.

Finally, I extend my deep appreciation to my family, friends, and well-wishers, whose constant motivation and support have been a source of strength in successfully completing this training and report.



Place: Patna

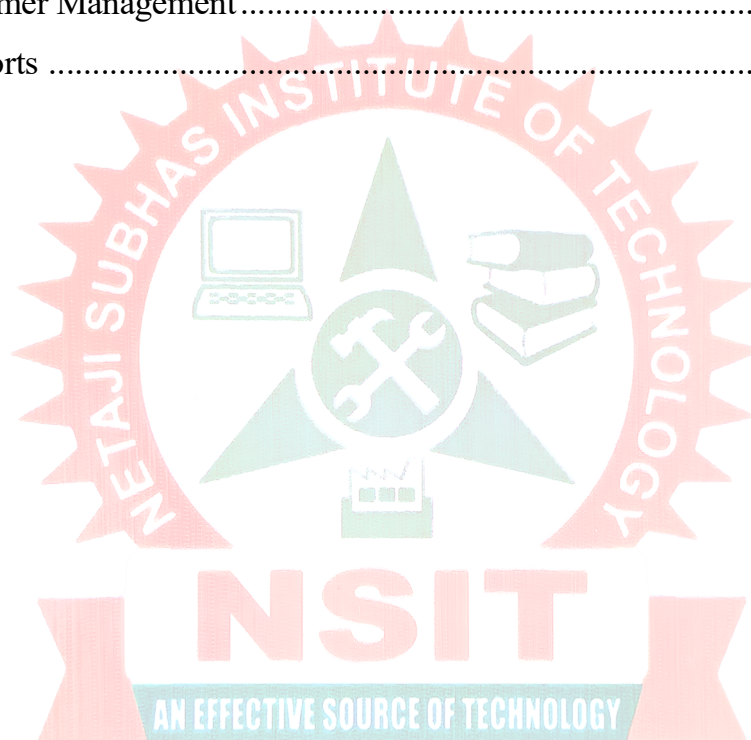
Student Name: Suraj Arya

Date:

Registration No.: 25105103904

List of Figures

Fig. No. 1 Group Photo of Training Participants at C-DAC Patna.....	04
Fig. No. 2 Initial Screen of Retail POS System (Console-Based).....	07
Fig. No. 3 Directory Structure.....	09
Fig. No. 4 Method Call Flow	09
Fig. No. 5 User Login.....	10
Fig. No. 6 Product Management	11
Fig. No. 7 Inventory Management	11
Fig. No. 8 Billing System	12
Fig. No. 9 Customer Management.....	12
Fig. No. 10 Reports	13



List of Abbreviations

- **OOP** – Object-Oriented Programming
- **JDK** – Java Development Kit
- **JVM** – Java Virtual Machine
- **JRE** – Java Runtime Environment
- **SDLC** – Software Development Life Cycle
- **GUI** – Graphical User Interface
- **CLI** – Command Line Interface
- **API** – Application Programming Interface
- **DBMS** – Database Management System
- **IDE** – Integrated Development Environment
- **SQL** – Structured Query Language
- **CSV** – Comma-Separated Values
- **I/O** – Input/Output
- **CRUD** – Create, Read, Update, Delete
- **POS** – Point of Sale
- **RAM** – Random Access Memory
- **CPU** – Central Processing Unit

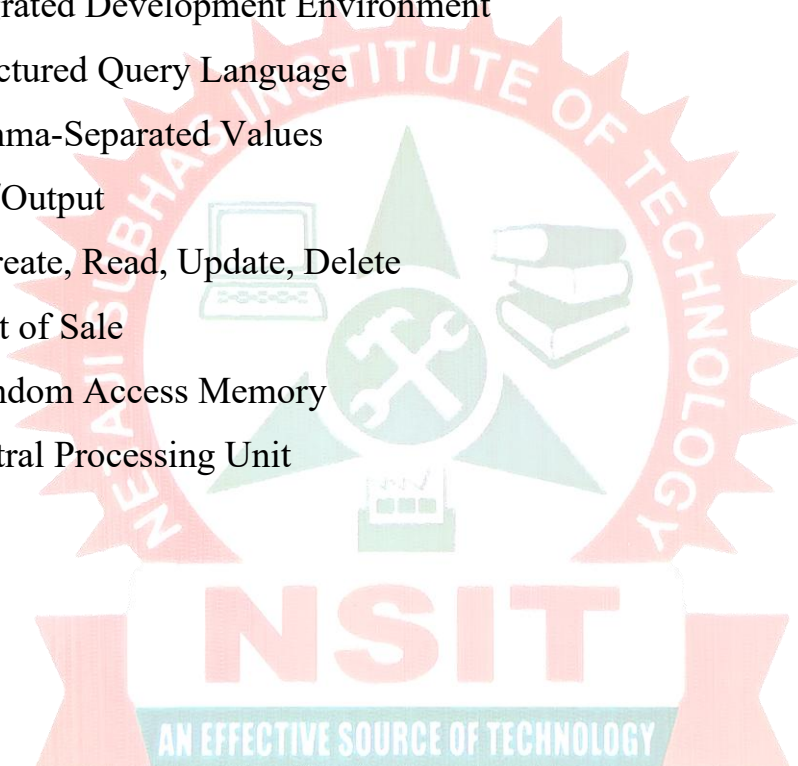
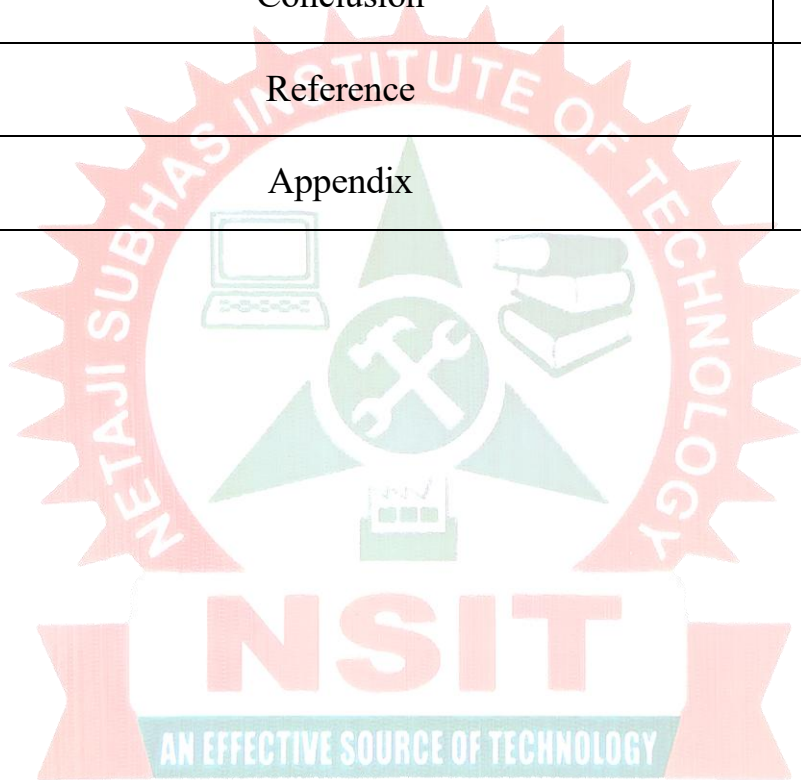


Table of Contents

S. No	Particulars	Page No.
1.	Introduction	1-4
2.	Training Details	5-6
3.	Projects	7-13
4.	Learning outcome	14
5.	Conclusion	15-16
6.	Reference	17
7.	Appendix	18-19



Certificate No.: C-DAC/PAT/2512/19020



CERTIFICATE OF COMPLETION

This is to certify that

Suraj Arya

PRN : STT-25128071920 has successfully completed
training cum internship in
OOP's concept using CoreJava
offered by C-DAC Patna
from 22.12.2025 to 02.01.2026 with grade **A⁺**

Ritesh R. Dhote

Scientist 'E' & HoD (ACTS)
C-DAC Patna

Grade Scale: A⁺ >=85%, A >=70% to <85%, B >=60% to <70%, C >=40% to <60%, Fail <40%

INTRODUCTION

The purpose of this Industrial Training – I was to bridge the gap between theoretical knowledge and practical implementation. In college, we learn many concepts, but this training helped me understand how these concepts are actually used in real-life situations.

This training aimed to provide hands-on experience in programming using Core Java. It helped me improve my coding skills and understand how to write clean and efficient programs. I also learned how different concepts of Object-Oriented Programming (OOP) like classes, objects, inheritance, and polymorphism are applied in real software development.

Another important purpose of this training was to develop problem-solving ability and logical thinking. By working on practical tasks and small programs, I gained confidence in coding and debugging errors.

Overall, this training helped me understand the basics of software development, improved my technical knowledge, and prepared me for future academic projects and industry work.

Objectives

The main objectives of the training were:

- To understand the fundamentals of **Java Programming Language**
- To learn and implement **Object-Oriented Programming (OOP) concepts**
- To gain practical knowledge of **Core Java features** such as classes, objects, inheritance, and polymorphism
- To develop problem-solving and coding skills
- To gain exposure to real-time project development
- To enhance technical and logical thinking abilities

Overview of Training Program

The Industrial Training – I was conducted at **C-DAC, Patna** for a duration of two weeks, from **22nd December 2025 to 3rd January 2026**. The training sessions were held daily from 9:00 AM to 1:30 PM.

The training focused on **“OOP’s Concepts using Core Java”** and included both theory and practical sessions. In the beginning, basic concepts of Java were taught, such as data types, operators, and control statements. After that, we learned important Object-Oriented Programming (OOP) concepts like classes, objects, inheritance, polymorphism, and encapsulation.

As the training progressed, more advanced topics were covered, including interfaces, exception handling, file handling, collections framework, and multithreading. Each topic was explained with examples and practical coding, which made it easier to understand.

The sessions were conducted by experienced faculty members, **Mr. Vaibhav Patle** and **Ms. Nishu**, who guided us throughout the training and helped us clear our doubts. At the end of the training, we worked on a small project and gave a final assessment to test our understanding.

Overall, the training program was well-structured and helped me gain both theoretical knowledge and practical experience in Java programming.

COMPANY PROFILE

Introduction to the Organization

The **Centre for Development of Advanced Computing (C-DAC)** is a well-known research and development organization in India. It works under the Ministry of Electronics and Information Technology (MeitY), Government of India.

C-DAC was established to develop advanced technologies in the field of computing. Over the years, it has played an important role in the growth of Information Technology in India. It is especially known for its work in supercomputing, software development, and technical education.

C-DAC has many centers across India, and each center focuses on research, development, and training. The Patna center mainly focuses on providing training programs and skill development for students.

Location

The training was conducted at **C-DAC, Patna**, located in Bihar, India. This center provides a good learning environment with proper labs, systems, and experienced faculty members.

Services and Areas of Work

C-DAC works in many advanced areas of technology. Some of the main services and fields are:

- **High-Performance Computing (HPC):** Development of supercomputers for research and scientific work
- **Software Development:** Designing and developing software solutions for various industries
- **Cyber Security:** Working on data protection, security systems, and ethical hacking
- **Artificial Intelligence (AI) & Machine Learning (ML):** Developing smart systems and automation technologies
- **Embedded Systems & IoT:** Working on hardware-based programming and smart devices
- **Education and Training:** Providing various training programs for students and professionals

C-DAC is also known for conducting short-term and long-term courses that help students improve their practical skills and prepare for the IT industry.

Role in Education and Training

One of the important roles of C-DAC is to provide quality training to students. It organizes different training programs like programming, software development, and new technologies.

During my training, I attended a short-term course on “**OOP’s Concepts using Core Java**”. The training included both theory and practical sessions, which helped me understand the concepts clearly.

The training programs at C-DAC focus on real-world applications, which makes students industry-ready.

Organizational Structure

C-DAC follows a proper organizational structure to manage its work smoothly. The structure includes:

- **Director General:** Overall head of C-DAC
- **Centre Director:** Head of a specific center (like Patna)
- **Project Managers / Program Coordinators:** Manage projects and training programs
- **Faculty / Trainers:** Teach students and provide guidance
- **Technical Staff:** Help in lab work and system management

During the training, the sessions were conducted by **Mr. Vaibhav Patle** and **Ms. Nishu**, who guided us in both theory and practical work.



Fig. 1: Group Photo of Training Participants at C-DAC Patna

TRAINING DETAILS

Duration of Training

The Industrial Training – I was conducted for a period of **two weeks**, from **22nd December 2025 to 3rd January 2026** at C-DAC, Patna.

The training sessions were held daily from **09:00 AM to 01:30 PM**.

This duration was very helpful to understand both basic and advanced concepts of Java programming in a structured way.

Department

The training was conducted under the **Software Development / Training Department** of C-DAC, Patna.

This department focuses on providing technical knowledge and practical skills to students in the field of programming and software development. It also helps students to understand how real-world software systems are developed.

Tools and Technologies Used

During the training, I worked with different tools and technologies related to Core Java. These include:

- **Core Java (JDK):** Used for writing and running Java programs
- **Object-Oriented Programming (OOP):** Concepts like class, object, inheritance, polymorphism, etc.
- **Java Collections Framework:** Used to store and manage data
- **Exception Handling:** To handle runtime errors in programs
- **File Handling:** For reading and writing data in files
- **Multithreading:** To perform multiple tasks at the same time
- **IDE (Eclipse / IntelliJ IDEA):** Used to write and execute Java programs easily

These tools helped me gain practical knowledge and improve my coding skills.

Training Methodology

The training program was designed in a simple and effective way. Each topic was first explained in theory, and then practical examples were given.

- Trainers explained concepts using real-life examples
- After explanation, coding was done step-by-step
- Students were encouraged to practice on their own
- Doubts were cleared regularly

Daily / Weekly Activities

Week 1 (22nd December – 27th December 2025)

In the first week, the focus was on basic concepts of Java:

- Introduction to Java Programming Language
- Data Types and Operators
- Flow Control Statements (if-else, loops)
- OOP Concepts (Class and Object)
- Constructors and Parameter Passing
- Inheritance in Java
- Abstract Classes and Interfaces
- Packages and Import Statements
- Introduction to Arrays

This week helped me build a strong foundation in Java.

Week 2 (29th December 2025 – 3rd January 2026)

In the second week, more advanced topics were covered:

- Wrapper Classes and String Handling
- Exception Handling
- File Handling
- Collections Framework
- Multithreading and Inner Classes
- Project Work
- Final Assessment

During this week, I got practical exposure and worked on small programs and a project.

Overall Experience

The training experience at C-DAC Patna was very good. The sessions were interactive, and the trainers were supportive. I was able to learn new concepts and improve my practical skills.

This training helped me become more confident in Java programming and prepared me for future academic projects and real-world applications.

PROJECT WORK

Project Title

Retail POS System (Console-Based Application using Core Java)

```
PS E:\Btech\CDAC-JAVA-2w\Retail-POS-System> & 'C:\Program Files\Java\jdk-26\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\offic\AppData\Roaming\Code\User\workspaceStorage\319c212f3d342d25ec7b31e79ec86997\redhat.java\jdt_ws\Retail-POS-System_b804cbda\bin' 'com.retailpos.main.MainApp'
```

```
=====
```

RETAIL

POS SYSTEM v1.0

```
=====
```

Developed by Team of RetailRangers
(Suraj Arya, Aryan Pandey, Vaivabhai Anand, Atul Mishra)

```
=====
```

Fig. 2: Initial Screen of Retail POS System (Console-Based)

Introduction to the Project

During the training, we developed a **Retail Point of Sale (POS) System** using Core Java. It is a **console-based application**, which means it runs on the command prompt and does not use any graphical interface. The system works in a menu-driven way where users can select options and perform different operations easily.

The main purpose of this project is to manage retail shop activities such as product management, billing, customer details, and inventory. Instead of using a database, all the data is stored in simple text files. This helped us understand file handling in Java and how data can be saved and managed without using complex systems.

This project helped us understand how real-world applications are developed using Java. We learned how to apply OOP concepts, file handling, collections, and exception handling in one project. It also improved our coding skills, problem-solving ability, and confidence in developing real applications.

Objective of the Project

The main objectives of this project were:

- To apply **Core Java and OOP concepts** in a real project
- To understand **file handling** for storing data
- To create a simple **billing and inventory system**
- To improve **coding and problem-solving skills**
- To learn how to work in a **team environment**

Team Structure

This project was developed by a team of 4 members, where each member had a specific role and responsibility. The work was divided into different modules so that the project could be completed in an organized and efficient way.

- **Suraj Arya (PRN: STT-25128071920):**
I worked on the **main application (MainApp.java)**, which controls the overall flow of the program. I created the menu-driven interface where users can select different options like product management, billing, and reports. I also handled user login and authentication. My main role was to connect all modules together and ensure the system runs smoothly.
- **Aryan Pandey (PRN: STT-25128071902):**
He worked on the **model classes** such as Product, Customer, and Bill. These classes are used to store and manage data in the program. He implemented encapsulation using private variables and getter/setter methods and designed the data structure of the system.
- **Vaibhavi Anand (PRN: STT-25128071922):**
She handled the **business logic part** of the project, including inventory management and billing system. She implemented features like adding/updating products, managing stock, and calculating bill amounts with GST.
- **Atul Anand Mishra (PRN: STT-25128071903):**
He worked on **file handling and exception handling**. He managed reading and writing data from text files and created custom exceptions to handle errors like invalid input and insufficient stock.

Working in a team helped us understand how real software projects are developed in parts. Each member focused on their task, and finally all modules were combined to build the complete system. This improved our teamwork, coordination, and practical development skills.

Project Structure

The project follows a proper modular structure where different functionalities are divided into packages. This makes the system easy to understand, manage, and maintain. Each package has a specific role, and all parts are connected together to form the complete application.

Directory Structure:

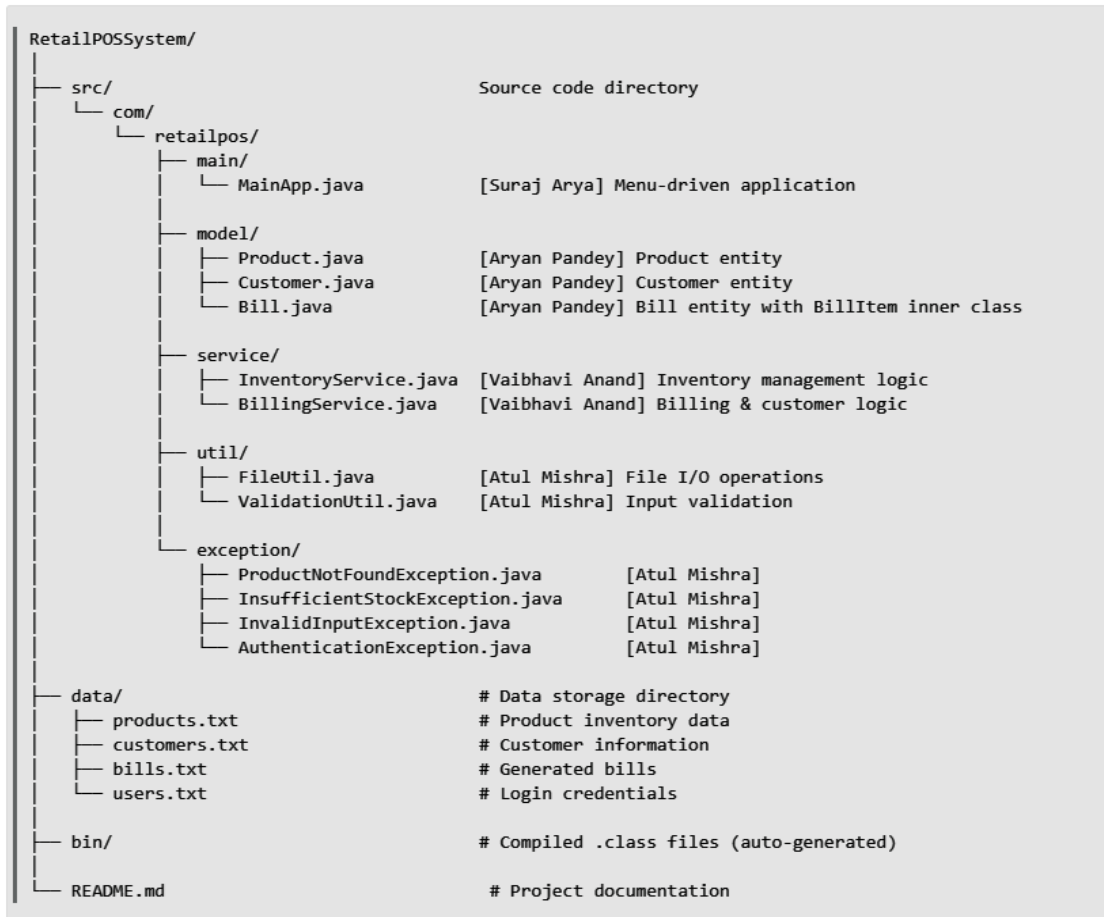


Fig. 3: Directory Structure

Method Call Flow:



Fig. 4: Method Call Flow

Technologies Used

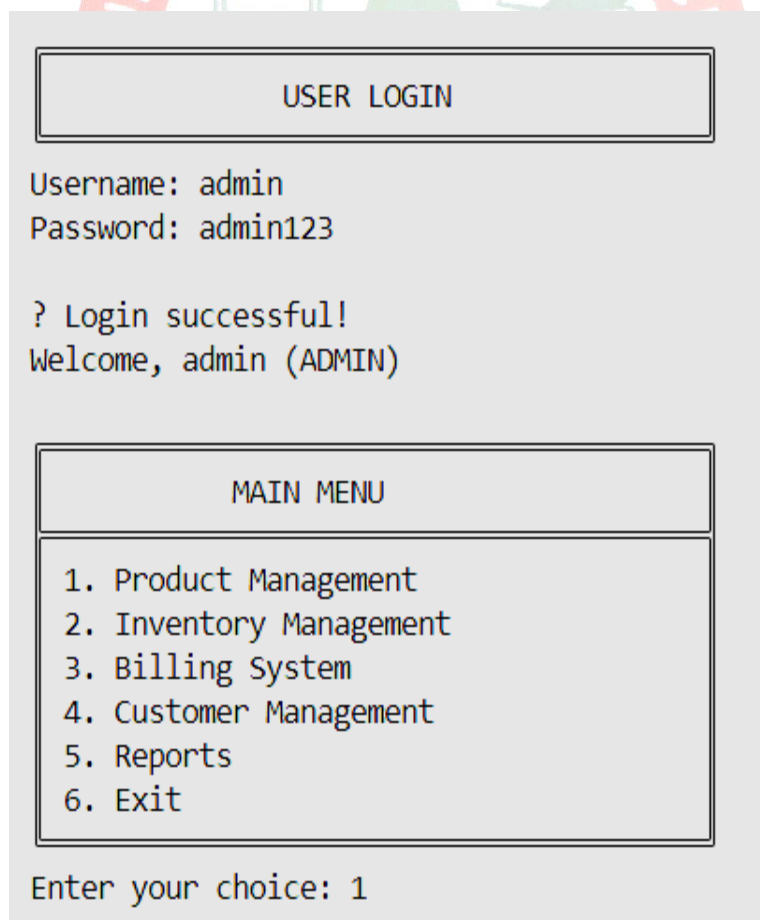
- Core Java
- Object-Oriented Programming (OOP)
- File Handling (Text Files)
- Collections Framework (ArrayList, HashMap)
- Exception Handling
- IDE (Eclipse / IntelliJ IDEA)

System Features

The system provides different features that help in managing retail shop operations in an easy and efficient way. These features make the application useful and user-friendly.

✓ User Login System

- Login using username and password
- Different roles like Admin and Cashier



```
USER LOGIN

Username: admin
Password: admin123

? Login successful!
Welcome, admin (ADMIN)

MAIN MENU

1. Product Management
2. Inventory Management
3. Billing System
4. Customer Management
5. Reports
6. Exit

Enter your choice: 1
```

Fig. 5: User Login

✓ Product Management

- Add, update, delete products
- Search and view product details

PRODUCT MANAGEMENT

1. Add New Product
 2. View All Products
 3. Search Product
 4. Update Product
 5. Delete Product
 6. Back to Main Menu

Enter your choice: 2

INVENTORY LIST

ID	Product Name	Category	Price	Stock
P005	Headphones	Electronics	Rs.2500.00	25
P004	Monitor	Electronics	Rs.8500.00	20
P007	USB Cable	Accessories	Rs.150.00	100
P006	Printer	Electronics	Rs.12000.00	10
P001	Laptop	Electronics	Rs.45000.00	14
P003	Keyboard	Electronics	Rs.1200.00	30
P002	Mouse	Electronics	Rs.500.00	49
P010	Backpack	Accessories	Rs.1500.00	35
P009	Charger	Accessories	Rs.800.00	45
P008	Phone Cover	Accessories	Rs.300.00	80

Total Products: 10

Fig. 6: Product Management

✓ Inventory Management

- Check stock
- Update stock
- Low stock alert

INVENTORY MANAGEMENT

1. Check Stock
 2. Add Stock
 3. Low Stock Alert
 4. Inventory Value
 5. Back to Main Menu

Fig. 7: Inventory Management

✓ Billing System

- Create bills
- Add multiple products
- Calculate total amount with GST (18%)

BILLING SYSTEM

1. Create New Bill
2. View All Bills
3. Search Bill
4. Back to Main Menu

Enter your choice: 2

BILLS HISTORY

Bill ID	Cust ID	Customer Name	Date	Amount
B0001	C001	Aryan Pandey	01-01-2026 23:23:49	Rs.53690.00

Total Bills: 1

Fig. 8: Billing System

✓ Customer Management

- Add and view customers
- Track customer purchases

CUSTOMER MANAGEMENT

1. Add New Customer
2. View All Customers
3. Search Customer
4. Back to Main Menu

Enter your choice: 2

CUSTOMER LIST

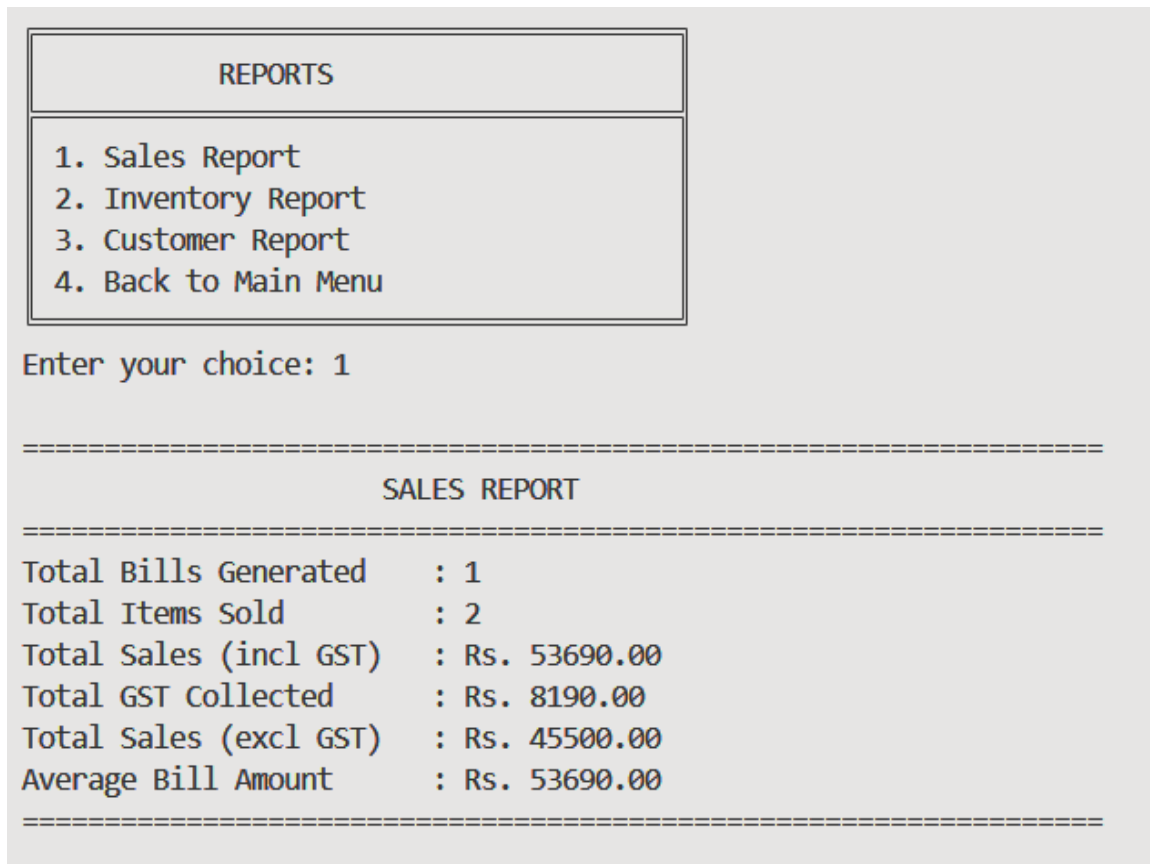
ID	Name	Phone	Email	Total Spent
C002	Rahul Sharma	9876543210	rahul.sharma@email.com	Rs.0.00
C001	Aryan Pandey	7410741010	Aryan@gmail.com	Rs.53690.00
C006	Amit Kumar	9876543212	amit.kumar@email.com	Rs.0.00
C005	Vijay Reddy	9876543214	vijay.reddy@email.com	Rs.0.00
C004	Sneha Patel	9876543213	sneha.patel@email.com	Rs.0.00
C003	Priya Singh	9876543211	priya.singh@email.com	Rs.0.00

Total Customers: 6

Fig.9: Customer Management

✓ Reports

- Sales report
- Inventory report
- Customer report



The screenshot shows a Java application window titled "REPORTS". Inside, there is a menu with four options: "1. Sales Report", "2. Inventory Report", "3. Customer Report", and "4. Back to Main Menu". Below the menu, the user has entered "1" as their choice. The application then displays the "SALES REPORT" with the following data:

```

=====
                        SALES REPORT
=====
Total Bills Generated      : 1
Total Items Sold          : 2
Total Sales (incl GST)    : Rs. 53690.00
Total GST Collected      : Rs. 8190.00
Total Sales (excl GST)    : Rs. 45500.00
Average Bill Amount       : Rs. 53690.00
=====
  
```

Fig. 10: Reports

File Handling System

Instead of using a database, all data is stored in **text files**, such as:

- products.txt → Product details
- customers.txt → Customer details
- bills.txt → Billing records
- users.txt → Login credentials

This helped us understand how file handling works in Java.

Working of the Project

1. User logs into the system
2. Selects an option from the menu
3. Performs operations like adding products or creating bills
4. Data is stored or updated in text files
5. Output is displayed on the console

Learning Outcomes

Skills Gained

Through this project, several important technical and analytical skills were developed. Strong proficiency in **Java programming** was achieved, along with improved understanding of **Object-Oriented Programming (OOP)** concepts such as encapsulation, inheritance, and polymorphism. Skills in **problem-solving and logical thinking** were enhanced by designing features like billing systems and inventory management. Additionally, experience was gained in **debugging and error handling**, which improved the ability to write efficient and error-free code. Working with **file handling and collections** also strengthened data management skills.

Knowledge Acquired

This project helped in acquiring in-depth knowledge of how a complete software system is designed and implemented. A better understanding of **Java Collections Framework** (ArrayList, HashMap) was gained for efficient data storage and retrieval. Knowledge of **file handling techniques** was developed, enabling permanent data storage. The project also provided insights into **exception handling mechanisms** to ensure smooth program execution. Furthermore, it improved understanding of **modular programming and system design principles**, which are essential for building scalable applications.

Practical Exposure

The project provided valuable hands-on experience in developing a real-world application. It offered practical exposure to **software development life cycle (SDLC)**, including planning, designing, coding, testing, and debugging. Experience was gained in working with **real-time data processing**, user input handling, and system interaction. It also helped in understanding how different components of a system interact with each other. Overall, this project bridged the gap between theoretical concepts and practical implementation, preparing for future real-world development tasks.

Conclusion

Overall Experience

This project has been a highly enriching and insightful learning experience that contributed significantly to both technical growth and overall development. It provided an opportunity to practically implement theoretical concepts learned during the course, especially in **Java programming and Object-Oriented Programming (OOP)**. By working on this project, a deeper understanding of concepts such as **encapsulation, inheritance, polymorphism, abstraction, collections, file handling, and exception handling** was achieved.

The entire process of developing the project—from requirement analysis, designing the structure, coding, testing, to debugging—helped in understanding how real-world applications are built. It improved the ability to break complex problems into smaller modules and solve them step by step. Working on features like **data management, billing system, inventory control, and user interaction** gave practical exposure to real-time application scenarios.

During the project, several challenges were faced, such as handling errors, managing data efficiently, and ensuring proper program flow. Overcoming these challenges helped in improving **problem-solving skills, logical thinking, and debugging capabilities**. It also enhanced the ability to write clean, efficient, and well-structured code.

Additionally, this project became easier and more understandable for me because I had already studied **Java during my diploma** and had good hands-on practice in it. Due to my previous experience and consistent practice, I was able to understand and implement concepts more effectively. This prior knowledge also helped me in completing tasks efficiently and with better accuracy. In my diploma, I also achieved a **2nd rank in class with an A+ grade**, which further reflects my strong foundation and interest in this subject.

Overall, this experience has been extremely valuable in bridging the gap between academic knowledge and practical implementation, making it a strong foundation for future software development work.

Importance of Training

Industrial training is an essential part of an engineering curriculum as it provides real-world exposure and practical knowledge that cannot be fully gained through classroom learning alone. It plays a vital role in developing technical expertise and understanding how concepts are applied in real industry environments.

Through training, students gain hands-on experience in working with technologies, tools, and frameworks used in the industry. It helps in understanding the **Software Development Life Cycle (SDLC)**, including planning, designing, coding, testing, and deployment. This practical exposure enhances the ability to work on real-time projects and prepares students for professional challenges.

Training also helps in developing important soft skills such as **communication, teamwork, time management, and problem-solving**. Working in a structured environment improves discipline and teaches how to meet deadlines and work under pressure. It also builds confidence and prepares students for interviews and job roles.

Another important aspect of training is learning **industry standards and coding practices**, which are essential for developing scalable and maintainable software. It helps in understanding how professional developers write code, collaborate, and manage projects efficiently.

Overall, industrial training is crucial for shaping a student's career. It provides a platform to apply theoretical knowledge, gain practical experience, and develop the skills required to become a competent software developer. This experience not only enhances technical abilities but also builds confidence and prepares for future career opportunities in the field of technology.

References

- Official Java Documentation: <https://docs.oracle.com/javase/>
- GeeksforGeeks: <https://www.geeksforgeeks.org/java/>
- W3Schools Java Tutorial: <https://www.w3schools.com/java/>
- TutorialsPoint Java Guide: <https://www.tutorialspoint.com/java/>
- Stack Overflow (for problem-solving and debugging support):
<https://stackoverflow.com/>
- Class notes and lecture materials provided during academic sessions
- Practical lab manuals and assignments
- Online video lectures and tutorials on Java programming
- Previous knowledge and hands-on practice gained during diploma studies



Code snippets

```

MainApp.java •
tailPOSSystem > src > com > retailpos > main > J MainApp.java >  MainApp

package com.retailpos.main;

import com.retailpos.model.Product;
import com.retailpos.model.Customer;
import com.retailpos.model.Bill;
import com.retailpos.service.InventoryService;
import com.retailpos.service.BillingService;
import com.retailpos.exception.*;
import com.retailpos.util.IdUtil;
import com.retailpos.util.ValidationUtil;
import java.util.Scanner;
import java.util.ArrayList;
import java.io.IOException;

public class Mainapp {

    private static Scanner scanner = new Scanner(System.in);
    private static InventoryService inventoryService;
    private static BillingService billingService;
    private static String currentUser;
    private static String currentUserRole;

    Run (Debug)
    public static void main(String[] args) {

        // Initialize files
        #100111.initializedFiles();

        // Initialize services
        inventoryService = new InventoryService();
        billingService = new BillingService(inventoryService);

        // Display welcome screen
        displayWelcomeScreen();

        // User authentication
        try {
            if (authenticateUser()) {
                System.out.println("Authentication failed. Exiting...");
                return;
            }
        } catch (AuthenticationException e) {
            System.out.println("X " + e.getMessage());
            return;
        }

        // Main menu loop
        boolean running = true;
        while (running) {
            try {
                displayMainMenu();
                String choice = scanner.nextLine().trim();

                switch (choice) {
                    case "1":
                        productManagementMenu();
                        break;
                    case "2":
                        inventoryManagementMenu();
                        break;
                    case "3":
                        billingMenu();
                        break;
                    case "4":
                        customerManagementMenu();
                        break;
                    case "5":
                        reportMenu();
                        break;
                    case "6":
                        running = false;
                        exitApplication();
                        break;
                    default:
                        System.out.println("X Invalid choice. Please try again.");
                }
            } catch (Exception e) {
                System.out.println("X Error: " + e.getMessage());
            }
        }
    }
}

```

in 0 0 Java: Ready

File Edit Selection View Go Run Terminal Help

```

7 J MainApp.java •
15 RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp
169 public class MainApp {
170     // Product Management Menu
171     //
172     private static void prodManagementMenu() {
173         boolean back = false;
174
175         while (back) {
176             System.out.println("\n");
177             System.out.println("PRODUCT MANAGEMENT") *";
178             System.out.println("\n");
179             System.out.println("1. Add New Product") *";
180             System.out.println("2. View All Products") *";
181             System.out.println("3. Search Product") *";
182             System.out.println("4. Update Product") *";
183             System.out.println("5. Delete Product") *";
184             System.out.println("6. Back to Main Menu") *";
185             System.out.println("\n");
186             System.out.println("Enter your choice:"); *";
187
188             String choice = scanner.nextLine().trim();
189
190             try {
191                 switch (choice) {
192                     case "1":
193                         addProduct();
194                         break;
195                     case "2":
196                         InventoryService.viewallProducts();
197                         break;
198                     case "3":
199                         searchProduct();
200                         break;
201                     case "4":
202                         updateProduct();
203                         break;
204                     case "5":
205                         deleteProduct();
206                         break;
207                     case "6":
208                         back = true;
209                         break;
210                     default:
211                         System.out.println("X Invalid choice.");
212
213                 }
214             } catch (Exception e) {
215                 System.out.println("X Error: " + e.getMessage());
216             }
217         }
218     }
219
220     /**
221     * Add new product
222     */
223     private static void addProduct() {
224         try {
225             System.out.println("\nAdd New Product ---");
226
227             String productid = InventoryService.generateNextProductid();
228             System.out.println("Product ID (Auto-generated): " + productid);
229
230             System.out.print("Product Name: ");
231             String name = scanner.nextLine().trim();
232             ValidationResult.validateNotNull(name, fieldName: "Product Name");
233
234             System.out.print("Category: ");
235             String category = scanner.nextLine().trim();
236             ValidationResult.validateNotNull(category, fieldName: "Category");
237
238             System.out.print("Price: ");
239             String priceStr = scanner.nextLine().trim();
240             double price = ValidationResult.validatePositiveDouble(priceStr, fieldName: "Price");
241
242             String stockStr = scanner.nextLine().trim();
243             int stock = ValidationResult.validatePositiveInteger(stockStr, fieldName: "Stock Quantity");
244
245             System.out.print("Description: ");
246             String description = scanner.nextLine().trim();
247
248             Product product = new Product(name, category, price, stock, description);
249             InventoryService.addProduct(product);
250
251         } catch (InvalidInputException e) {
252             System.out.println(e.getMessage());
253         }
254     }
255 }

```

main 0 0 Java: Ready

File Edit Selection View Go Run Terminal Help

[illegible]

< main ↺ ⊗ 0 ⚠ 0 🔍 ☕ Java: Ready

System | le | Edit | Selection | View | Go | Run | Terminal | Help

```

J MainApp.java ●
215 public class MainApp {
216     ~ Search product
217     ~ ~
218     private static void searchProduct() {
219         System.out.println("Enter search term (ID/Name/Category): ");
220         String searchTerm = scanner.nextLine().trim();
221
222         ArrayList<Product> results = inventoryService.searchProduct(searchTerm);
223
224         if (results.isEmpty()) {
225             System.out.println("No products found.");
226         } else {
227             System.out.println("----- Search Results -----");
228             for (Product product : results) {
229                 product.displayProduct();
230             }
231         }
232     }
233
234     /**
235      * Update product
236      */
237     private static void updateProduct() {
238         try {
239             System.out.println("Enter Product ID to update: ");
240             String productId = scanner.nextLine().trim();
241
242             Product existingProduct = inventoryService.getProduct(productId);
243             existingProduct.displayProduct();
244
245             System.out.println("Enter new details (press Enter to keep current value):");
246
247             System.out.print("Product Name (" + existingProduct.getProductName() + "): ");
248             String newName = scanner.nextLine().trim();
249             if (newName.isEmpty()) {
250                 existingProduct.setProductName(name);
251             }
252
253             System.out.print("Category (" + existingProduct.getCategory() + "): ");
254             String newCategory = scanner.nextLine().trim();
255             if (newCategory.isEmpty()) {
256                 existingProduct.setCategory(category);
257             }
258
259             System.out.print("Price (" + existingProduct.getPrice() + "): ");
260             String newPrice = scanner.nextLine().trim();
261             if (newPrice.isEmpty()) {
262                 double price = ValidationUtil.validatePositiveDouble(prompter, fieldName: "Price");
263                 existingProduct.setPrice(price);
264             }
265
266             System.out.print("Stock Quantity (" + existingProduct.getStockQuantity() + "): ");
267             String newStock = scanner.nextLine().trim();
268             if (newStock.isEmpty()) {
269                 int stock = ValidationUtil.validatePositiveInteger(prompter, fieldName: "Stock");
270                 existingProduct.setStockQuantity(stock);
271             }
272
273             System.out.print("Description (" + existingProduct.getDescription() + "): ");
274             String newDescription = scanner.nextLine().trim();
275             if (newDescription.isEmpty()) {
276                 existingProduct.setDescription(description);
277             }
278
279             inventoryService.updateProduct(productId, existingProduct);
280
281             catch (ProductNameException | InvalidInputException e) {
282                 System.out.println("X " + e.getMessage());
283             }
284         }
285     }
286
287     /**
288      * Delete product
289      */
290     private static void deleteProduct() {
291         try {
292             System.out.println("Enter Product ID to delete: ");
293             String productId = scanner.nextLine().trim();
294
295             Product product = inventoryService.getProduct(productId);
296
297             System.out.println("Are you sure you want to delete this product? (yes/no): ");
298
299             if (Boolean.parseBoolean(scanner.nextLine().trim().toLowerCase())

```

```

337         if (confirm.equals(anObject: "yes"))

```

File Edit Selection View Go Run Terminal Help

← → Retail-POS-System File Edit Selection View Go Run Terminal Help

← → Retail-POS-System

```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void deleteProduct() {
17         Scanner scanner = new Scanner(System.in);
18         System.out.println("X * * * *");
19     }
20 }
21
22 /**
23  * Inventory Management Menu
24  */
25 private static void inventoryManagementMenu() {
26     boolean back = false;
27
28     while (!back) {
29         System.out.println("\n===== INVENTORY MANAGEMENT =====");
30         System.out.println("1. Check Stock");
31         System.out.println("2. Add Stock");
32         System.out.println("3. Low Stock Alert");
33         System.out.println("4. Inventory Value");
34         System.out.println("5. Back to Main Menu");
35         System.out.print("Enter your choice: ");
36         String choice = scanner.nextLine().trim();
37
38         try {
39             switch (choice) {
40                 case "1":
41                     checkStock();
42                     break;
43                 case "2":
44                     addStock();
45                     break;
46                 case "3":
47                     inventoryService.displayLowStockAlert();
48                     break;
49                 case "4":
50                     displayInventoryValue();
51                     break;
52                 case "5":
53                     back = true;
54                     break;
55                 default:
56                     System.out.println("X Invalid choice.");
57             }
58         } catch (Exception e) {
59             System.out.println("X Error: " + e.getMessage());
60         }
61     }
62 }
63
64 /**
65  * Check stock for a product
66  */
67 private static void checkStock() {
68     try {
69         System.out.print("Enter Product ID: ");
70         String productId = scanner.nextLine().trim();
71
72         Product product = inventoryService.getProduct(productId);
73
74         System.out.println("\n--- Stock Information ---");
75         System.out.println("Product ID : " + product.getId());
76         System.out.println("Product Name : " + product.getName());
77         System.out.println("Available Stock : " + product.getStock());
78         System.out.println("Status : " + (product.isInStock() ? "In Stock" : "Out of Stock"));
79     } catch (ProductNotFoundException e) {
80         System.out.println("X * " + e.getMessage());
81     }
82 }
83
84 /**
85  * Add stock to a product
86  */
87 private static void addStock() {
88     try {
89         System.out.print("Enter Product ID: ");
90         String productId = scanner.nextLine().trim();
91
92         Product product = inventoryService.getProduct(productId);
93     }
94 }
95
96 main Java: Ready

```



File Edit Selection View Go Run Terminal Help

← → Retail-POS-System File Edit Selection View Go Run Terminal Help

← → Retail-POS-System

```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void addCustomer() {
17         Scanner scanner = new Scanner(System.in);
18         String phone = scanner.nextLine().trim();
19         ValidationUtil.validatePhoneNumber(phone);
20
21         System.out.print("Email: ");
22         String email = scanner.nextLine().trim();
23         ValidationUtil.validateEmail(email);
24
25         Customer customer = new Customer(customerId, name, phone, email);
26         billingService.addCustomer(customer);
27     } catch (InvalidInputException e) {
28         System.out.println("X * " + e.getMessage());
29     }
30 }
31
32 private static void searchCustomer() {
33     System.out.print("Enter search term (ID/Name/Phone): ");
34     String searchTerm = scanner.nextLine().trim();
35
36     ArrayList<Customer> results = billingService.searchCustomers(searchTerm);
37
38     if (results.isEmpty()) {
39         System.out.println("No customers found.");
40     } else {
41         System.out.println("\n--- Search Results ---");
42         for (Customer customer : results) {
43             customer.displayCustomer();
44         }
45     }
46 }
47
48 /**
49  * Reports Menu
50  */
51 private static void reportsMenu() {
52     boolean back = false;
53
54     while (!back) {
55         System.out.println("\n===== REPORTS =====");
56         System.out.println("1. Sales Report");
57         System.out.println("2. Inventory Report");
58         System.out.println("3. Customer Report");
59         System.out.println("4. Back to Main Menu");
60         System.out.print("Enter your choice: ");
61         String choice = scanner.nextLine().trim();
62
63         try {
64             switch (choice) {
65                 case "1":
66                     billingService.generateSalesReport();
67                     break;
68                 case "2":
69                     inventoryService.viewAllProducts();
70                     displayInventoryValue();
71                     break;
72                 case "3":
73                     billingService.viewAllCustomers();
74                     break;
75                 case "4":
76                     back = true;
77                     break;
78                 default:
79                     System.out.println("X Invalid choice.");
80             }
81         } catch (Exception e) {
82             System.out.println("X Error: " + e.getMessage());
83         }
84     }
85 }
86
87 /**
88  * Exit application
89  */
90 private static void exitApplication() {
91     System.out.println("\n=====");
92     System.out.println("Thank you for using Retail POS System!");
93     System.out.println("Goodbye!");
94     scanner.close();
95 }
96
97 main Java: Ready

```



```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void addStock() {
17         Scanner scanner = new Scanner(System.in);
18         String productId = scanner.nextLine().trim();
19
20         Product product = inventoryService.getProduct(productId);
21         System.out.println("Current Stock: " + product.getStock());
22
23         System.out.print("Enter quantity to add: ");
24         String qtyStr = scanner.nextLine().trim();
25         int quantity = ValidationUtil.validatePositiveInteger(qtyStr, fieldName: "Quantity");
26         inventoryService.addStock(productId, quantity);
27
28         System.out.println("New Stock: " + inventoryService.getProduct(productId).getStock());
29     } catch (ProductNotFoundException | InvalidInputException e) {
30         System.out.println("X * " + e.getMessage());
31     }
32 }
33
34 /**
35  * Display total inventory value
36  */
37 private static void displayInventoryValue() {
38     double totalValue = inventoryService.getTotalInventoryValue();
39     int productCount = inventoryService.getProductCount();
40
41     System.out.println("\n--- Inventory Summary ---");
42     System.out.println("Total Products : " + productCount);
43     System.out.println("Total Inventory Value: Rs. " + String.format("%.2f", totalValue));
44 }
45
46 /**
47  * Billing Menu
48  */
49 private static void billingMenu() {
50     boolean back = false;
51
52     while (!back) {
53         System.out.println("\n===== BILLING SYSTEM =====");
54         System.out.println("1. Create New Bill");
55         System.out.println("2. View All Bills");
56         System.out.println("3. Search Bill");
57         System.out.println("4. Back to Main Menu");
58         System.out.print("Enter your choice: ");
59         String choice = scanner.nextLine().trim();
60
61         try {
62             switch (choice) {
63                 case "1":
64                     createBill();
65                     break;
66                 case "2":
67                     billingService.viewAllBills();
68                     break;
69                 case "3":
70                     searchBill();
71                     break;
72                 case "4":
73                     back = true;
74                     break;
75                 default:
76                     System.out.println("X Invalid choice.");
77             }
78         } catch (Exception e) {
79             System.out.println("X Error: " + e.getMessage());
80         }
81     }
82 }
83
84 /**
85  * Create new bill
86  */
87 private static void createBill() {
88     try {
89         System.out.print("\n--- Create New Bill ---");
90         System.out.print("Enter Customer ID: ");
91         String customerId = scanner.nextLine().trim();
92
93         while (addingItems == true) {
94             System.out.print("Enter Product ID (or 'done' to finish): ");
95             String productId = scanner.nextLine().trim();
96
97             if (productId.equals(scanner.nextLine().trim())) {
98                 break;
99             }
100             System.out.print("Enter Quantity: ");
101             String qtyStr = scanner.nextLine().trim();
102             int quantity = ValidationUtil.validatePositiveInteger(qtyStr, fieldName: "Quantity");
103
104             try {
105                 billingService.addToBill(productId, quantity);
106             } catch (ProductNotFoundException | InsufficientStockException e) {
107                 System.out.println("X * " + e.getMessage());
108                 System.out.print("Continue adding items? (yes/no): ");
109                 if (scanner.nextLine().trim().equalsIgnoreCase("yes")) {
110                     addingItems = false;
111                 }
112             }
113         }
114
115         if (bill.getTotalCount() > 0) {
116             System.out.print("Confirm billing? (yes/no): ");
117             String confirm = scanner.nextLine().trim().toLowerCase();
118
119             if (confirm.equals("yes")) {
120                 billingService.completeBilling(bill);
121             } else {
122                 System.out.println("Billing cancelled.");
123             }
124         } else {
125             System.out.println("No items added. Billing cancelled.");
126         }
127     } catch (InvalidInputException | ProductNotFoundException | InsufficientStockException e) {
128         System.out.println("X * " + e.getMessage());
129     }
130 }
131
132 /**
133  * Search bill
134  */
135 private static void searchBill() {
136     System.out.print("Enter search term (Bill ID/Customer ID/Name): ");
137     String searchTerm = scanner.nextLine().trim();
138
139     ArrayList<Bill> results = billingService.searchBills(searchTerm);
140
141     if (results.isEmpty()) {
142         System.out.println("No bills found.");
143     } else {
144         System.out.println("\n--- Search Results ---");
145         for (Bill bill : results) {
146             bill.displayInvoice();
147         }
148     }
149 }
150
151 /**
152  * Customer Management Menu
153  */
154 private static void customerManagementMenu() {
155     boolean back = false;
156
157     while (!back) {
158         System.out.println("\n===== CUSTOMER MANAGEMENT =====");
159         System.out.println("1. Add New Customer");
160         System.out.println("2. View All Customers");
161     }
162 }
163
164 main Java: Ready

```



File Edit Selection View Go Run Terminal Help

← → Retail-POS-System

```

J MainApp.java
RetailPOSSystem > src > com > retailpos > main > J MainApp.java > MainApp

15 public class MainApp {
16     private static void createBill() {
17         Scanner scanner = new Scanner(System.in);
18         System.out.print("\n--- Create New Bill ---");
19
20         System.out.print("Enter Customer ID: ");
21         String customerId = scanner.nextLine().trim();
22
23         Bill bill = billingService.createBill(customerId, currentUser);
24         System.out.println("Bill created: " + bill.getId());
25
26         boolean addingItems = true;
27         while (addingItems) {
28             System.out.print("Enter Product ID (or 'done' to finish): ");
29             String productId = scanner.nextLine().trim();
30
31             if (productId.equals(scanner.nextLine().trim())) {
32                 break;
33             }
34             System.out.print("Enter Quantity: ");
35             String qtyStr = scanner.nextLine().trim();
36             int quantity = ValidationUtil.validatePositiveInteger(qtyStr, fieldName: "Quantity");
37
38             try {
39                 billingService.addToBill(productId, quantity);
40             } catch (ProductNotFoundException | InsufficientStockException e) {
41                 System.out.println("X * " + e.getMessage());
42                 System.out.print("Continue adding items? (yes/no): ");
43                 if (scanner.nextLine().trim().equalsIgnoreCase("yes")) {
44                     addingItems = false;
45                 }
46             }
47         }
48
49         if (bill.getTotalCount() > 0) {
50             System.out.print("Confirm billing? (yes/no): ");
51             String confirm = scanner.nextLine().trim().toLowerCase();
52
53             if (confirm.equals("yes")) {
54                 billingService.completeBilling(bill);
55             } else {
56                 System.out.println("Billing cancelled.");
57             }
58         } else {
59             System.out.println("No items added. Billing cancelled.");
60         }
61     } catch (InvalidInputException | ProductNotFoundException | InsufficientStockException e) {
62         System.out.println("X * " + e.getMessage());
63     }
64 }
65
66 /**
67  * Search bill
68  */
69 private static void searchBill() {
70     System.out.print("Enter search term (Bill ID/Customer ID/Name): ");
71     String searchTerm = scanner.nextLine().trim();
72
73     ArrayList<Bill> results = billingService.searchBills(searchTerm);
74
75     if (results.isEmpty()) {
76         System.out.println("No bills found.");
77     } else {
78         System.out.println("\n--- Search Results ---");
79         for (Bill bill : results) {
80             bill.displayInvoice();
81         }
82     }
83 }
84
85 /**
86  * Customer Management Menu
87  */
88 private static void customerManagementMenu() {
89     boolean back = false;
90
91     while (!back) {
92         System.out.println("\n===== CUSTOMER MANAGEMENT =====");
93         System.out.println("1. Add New Customer");
94         System.out.println("2. View All Customers");
95     }
96 }
97
98 main Java: Ready

```

